## 1.Project

Train a language model from scratch, improve its architecture or training, and achieve the lowest reproducible test bits per byte within resource constraints. On the provided WikiText-2 benchmark and its fixed BPE-2048 tokenizer. Evaluate using FP32 and report a score that is reproducible on CPU.

## 2.Declaration of Alassistance

the student architecture in student.py: RMSNorm instead of LayerNorm, rotary position embeddings (RoPE) instead of learned absolute positions, a SwiGLU feed-forward block,residual-scaled initialisation, tied input/output embeddings, and the configuration switches used for the ablations;

the JSON configuration files under configs/, including the ablation configurations and the enlarged-baseline control configuration;

the checkpoint-selection logic added to `train.py`, which keeps the weights with the best validation BPB.

## 3.Structural optimization

Optimization suggestions proposed by AI are leveraged to optimize the code in student.py from three dimensions: normalization, positional information, and feed-forward layers. Main configuration and ablation configuration files under the `configs` directory are added to facilitate comparative analysis. Optimization for retaining the optimal weights is implemented in train.py. A total of eight training sessions, including those for testing, are conducted.

| project | modle | optimize | Effect |
|---|---|---|---|
| Normalization | LayerNorm (x-mean)/stdw+b | RMSNorm x/sqrt(mean($x^2$)+eps)w | Remove mean centering and bias to reduce computations while minimizing accuracy degradation. |
| Location information | Learning Absolute Position Embeddings | Rotary Position Embedding (RoPE, which rotates the query/key vectors according to their positions) | Rotate q and k such that the attention scores depend solely on the relative distance. |
| Feed-forward layer | GELU | SwiGLU | Adding gated linear units increases expressive power while maintaining similar parameter counts and floating-point operations. |

Normalization：LayerNorm→RMSNorm

For each position vector in the baseline, the mean is first subtracted, the result is then divided by the standard deviation, and finally a scaling and translation operation is performed ((x - mean)/std * w + b). In contrast, only division by the root mean square is performed, without mean subtraction or bias. The formulation is RMSNorm(x) = x / sqrt(mean($x^2$) + eps) * g, which eliminates one mean calculation and one bias addition, resulting in fewer parameters and lower inference cost.

Location information：Learnable Absolute Position → RoPE

The paradigm learns a vector for each of the 256 positions and adds it to the word embedding; instead of adding position vectors now, the query and key are "rotated" by an angle within the attention mechanism according to their positions: the position m is rotated by m·θ, and the position n is rotated by n·θ. The inner product of the two rotated vectors depends only on m−n, so the model naturally perceives the "relative distance", while the position information no longer occupies the parameter table.

Feed-forward layer：GELU（4x）→SwiGLU（8/3）

The hidden layer of the prototype is four times wider. Now there are three linear layers: one path serves as a "gate" (activated via SiLU), which is multiplied element-wise with the other path, and then projected back. The hidden width is set to 8/3 times rather than 4 times, exactly offsetting the additional matrix. The gating mechanism enables the network to modulate "how much to let through" on a per-channel basis, yielding stronger expressiveness than a single activation, while keeping the parameter count and floating-point operations nearly unchanged.

## 4.Ablation experiment

All configurations employ the same random seed and the same amount of training data. Only one factor is altered between each pair of matched groups: model scale (128/4 for the baseline versus 192/6) or a single mechanism (RoPE ↔ learnable position embedding, SwiGLU ↔ GELU, RMSNorm ↔ LayerNorm).

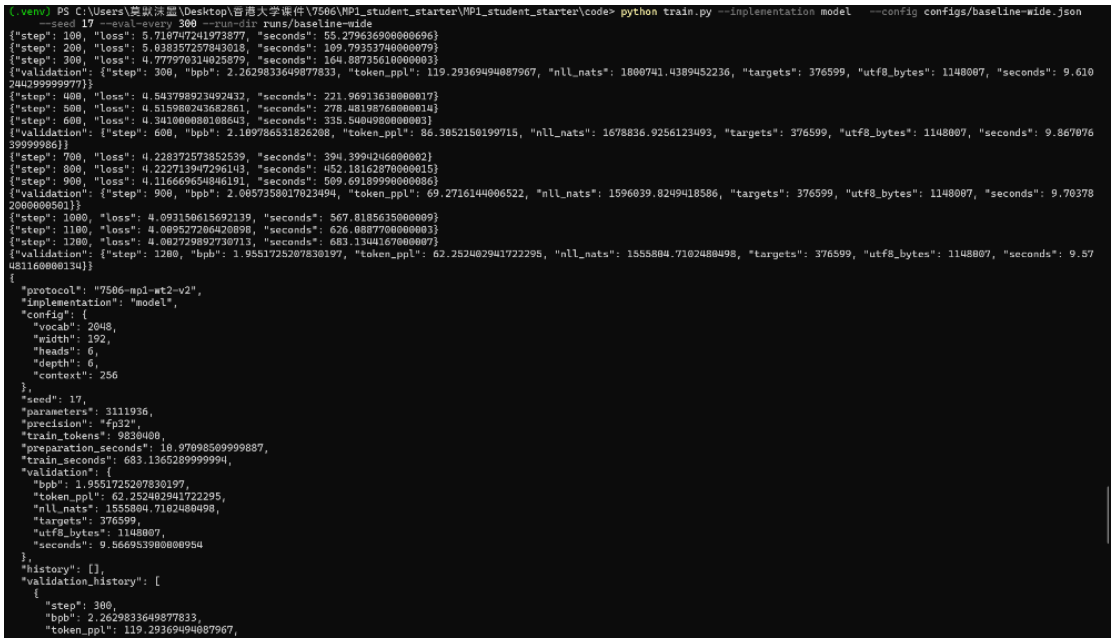



Figure1-baseline-wide

| Project | Structure | Number of parameters | BPB |
|---|---|---|---|
| Baseline | 128/4 | 1,088,256 | 2.0711 |
| baseline-wide | 192/6 | 3,111,936 | 1.9552 |

Table 1 - Contributions of Scale and Structure under the Same Training Budget

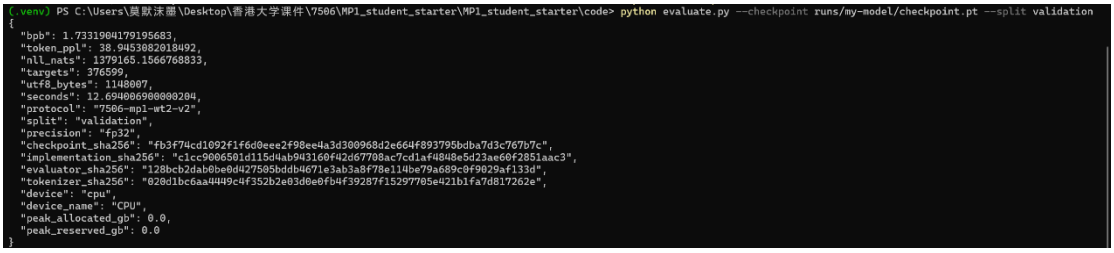



Figure 2 - Running results of the complete optimization model

```
  {
    "step": 300,
    "bpb": 2.204499445789840 45,
    "token_ppl": 105.42543033982344,
    "nll_nats": 1754199.6037978712,
    "targets": 376599,
    "utf8_bytes": 1148007,
    "seconds": 9.157868000000235
  },
  {
    "step": 600,
    "bpb": 2.0361032688850527,
    "token_ppl": 73.86214097797397,
    "nll_nats": 1620204.3669344147,
    "targets": 376599,
    "utf8_bytes": 1148007,
    "seconds": 9.77998709999974 4
  },
  {
    "step": 900,
    "bpb": 1.8988527437849276,
    "token_ppl": 55.26809625854684,
    "nll_nats": 1510988.933940668,
    "targets": 376599,
    "utf8_bytes": 1148007,
    "seconds": 9.552180699999553
  },
  {
    "step": 1200,
    "bpb": 1.8366156192431917,
    "token_ppl": 48.457707134806945,
    "nll_nats": 1461464.500426459,
    "targets": 376599,
    "utf8_bytes": 1148007,
    "seconds": 10.612604999999348
  }
],
"selected": {
  "step": 1200,
  "validation_bpb": 1.8366156192431917,
  "checkpoint_state": "final"
},
"intermediate_validation_seconds": 39.10264079999888,
"process_seconds": 861.1101140000004,
"torch_version": "2.7.1+cu126",
"threads": 4,
"checkpoint_sha256": "5475496be47b593bb1650a4b5594f32b63a83177550aad52adabcb552341291e",
"implementation_sha256": "c1cc9006501d115d4ab943160f42d67708ac7cd1af4848e5d23ae60f2851aac3",
"device": "cpu",
"device_name": "CPU",
"peak_allocated_gb": 0.0,
"peak_reserved_gb": 0.0
```


Figure 3. Running results without RoPE.


```
  {
    "step": 300,
    "bpb": 2.1115074615907967,
    "token_ppl": 86.61961380644622,
    "nll_nats": 1680206.33450353,
    "targets": 376599,
    "utf8_bytes": 1148007,
    "seconds": 11.430510099999992
  },
  {
    "step": 600,
    "bpb": 1.9391051674721176,
    "token_ppl": 60.174429063118374,
    "nll_nats": 1543019.309626569,
    "targets": 376599,
    "utf8_bytes": 1148007,
    "seconds": 11.936329999999966
  },
  {
    "step": 900,
    "bpb": 1.8524681674787 88,
    "token_ppl": 50.108325202635605,
    "nll_nats": 1474078.9725265976,
    "targets": 376599,
    "utf8_bytes": 1148007,
    "seconds": 11.61114600000019
  },
  {
    "step": 1200,
    "bpb": 1.8140632768906058,
    "token_ppl": 46.20274679478765,
    "nll_nats": 1443518.7487926194,
    "targets": 376599,
    "utf8_bytes": 1148007,
    "seconds": 11.593786899999941
  }
],
"selected": {
  "step": 1200,
  "validation_bpb": 1.8140632768906058,
  "checkpoint_state": "final"
},
"intermediate_validation_seconds": 46.57177210000009,
"process_seconds": 922.7610838999999,
"torch_version": "2.7.1+cu126",
"threads": 4,
"checkpoint_sha256": "af06fe5c74461df3f2babf991db4df2b15d8d1ad05953fe1edd02896bfb7b732",
"implementation_sha256": "c1cc9006501d115d4ab943160f42d67708ac7cd1af4848e5d23ae60f2851aac3",
"device": "cpu",
"device_name": "CPU",
"peak_allocated_gb": 0.0,
"peak_reserved_gb": 0.0
```


Figure 4 - Operation Results with SwiGLU Removed

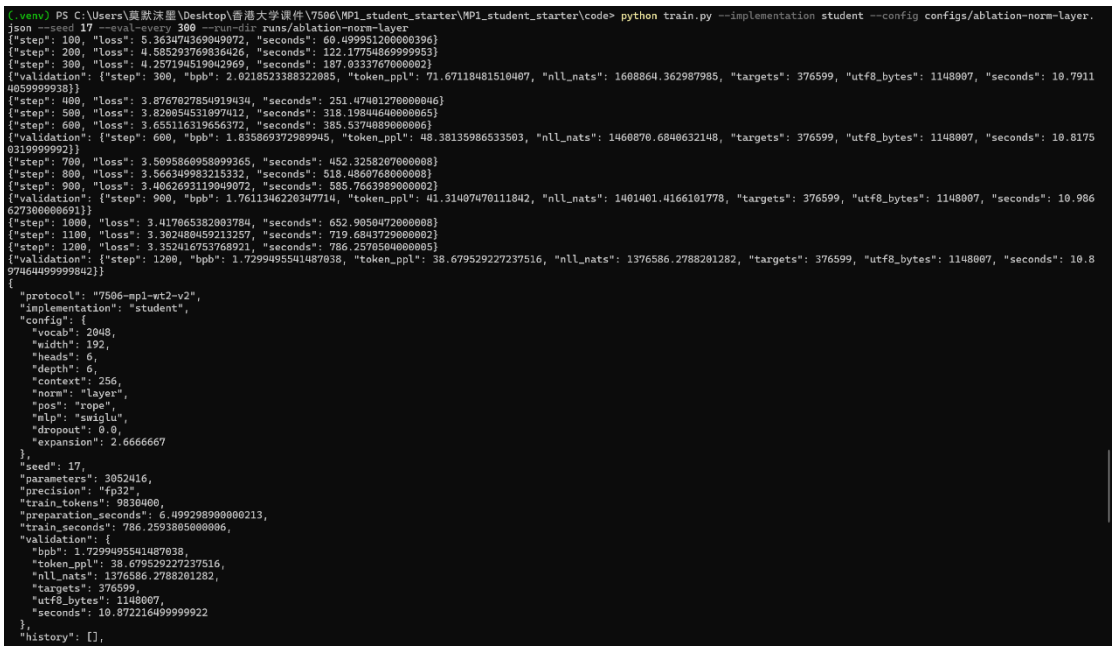



Figure 5. Operation results after removing RMSNorm

| Ablation | Set | Parameter count | BPB | Optimization effectiveness |
|---|---|---|---|---|
| complete model | RMSNorm+RoPE+SwiGLU | 3,049,920 | 1.7332 | — |
| RoPE | pos:learned | 3,099,072 | 1.8366 | +0.1034 |
| SwiGLU | mlp:gelu | 3,055,680 | 1.8141 | +0.0809 |
| RMSNorm | norm:layer | 3,052,416 | 1.7299 | −0.0032 |

Table 2 - Mechanism ablation (same training budget of 1,200 steps).

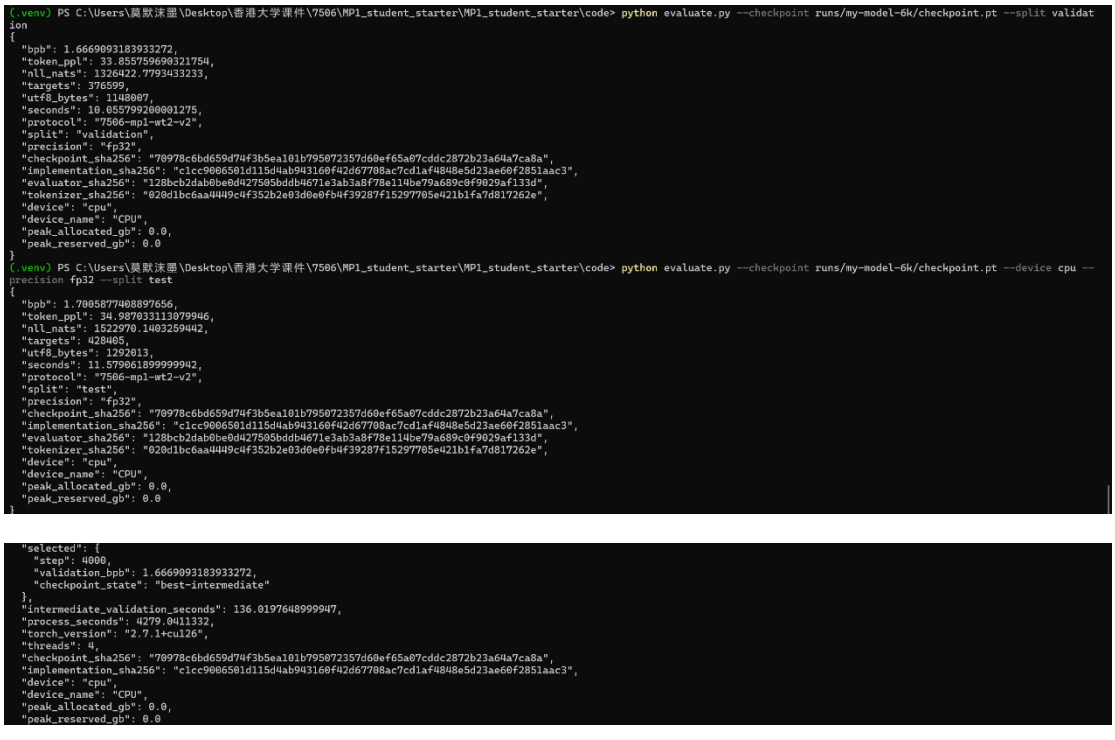



Figure 6 – Running results of 6,000 training steps under full optimization (the optimal score is obtained at step 4,000)

The optimal BPB of 1.6669 was achieved at step 4,000, after which it rebounded to 1.6712 by step 6,000, indicating that the model had already begun to overfit before the 6,000-step budget was exhausted.

The CPU scoring time is approximately 2.3 times that of the baseline; the peak memory is $\leqslant$ 4 GiB; the inference resources are $\leqslant$ 64 MiB.

## 5. Analysis of Results

Under a fixed training budget, scaling the baseline from 1.09M parameters to 3.11M parameters yields a reduction in BPB of 0.1159; at the same scale, replacing the architecture further reduces BPB by 0.2219.

After switching back from RMSNorm to LayerNorm, the validation BPB improved by 0.0032. This difference may stem from experimental fluctuations. It does not require computing the mean or adding a bias, involves fewer parameters, and incurs lower inference cost.

Removing RoPE incurs a loss of 0.1034 BPB, while removing SwiGLU incurs a loss of 0.0809 BPB. Positional encoding is the single component with the largest contribution.

The validation performance of BPB peaks at the 4000th step with a value of 1.6669, followed by a slight rebound to 1.6712 at the 6000th step.